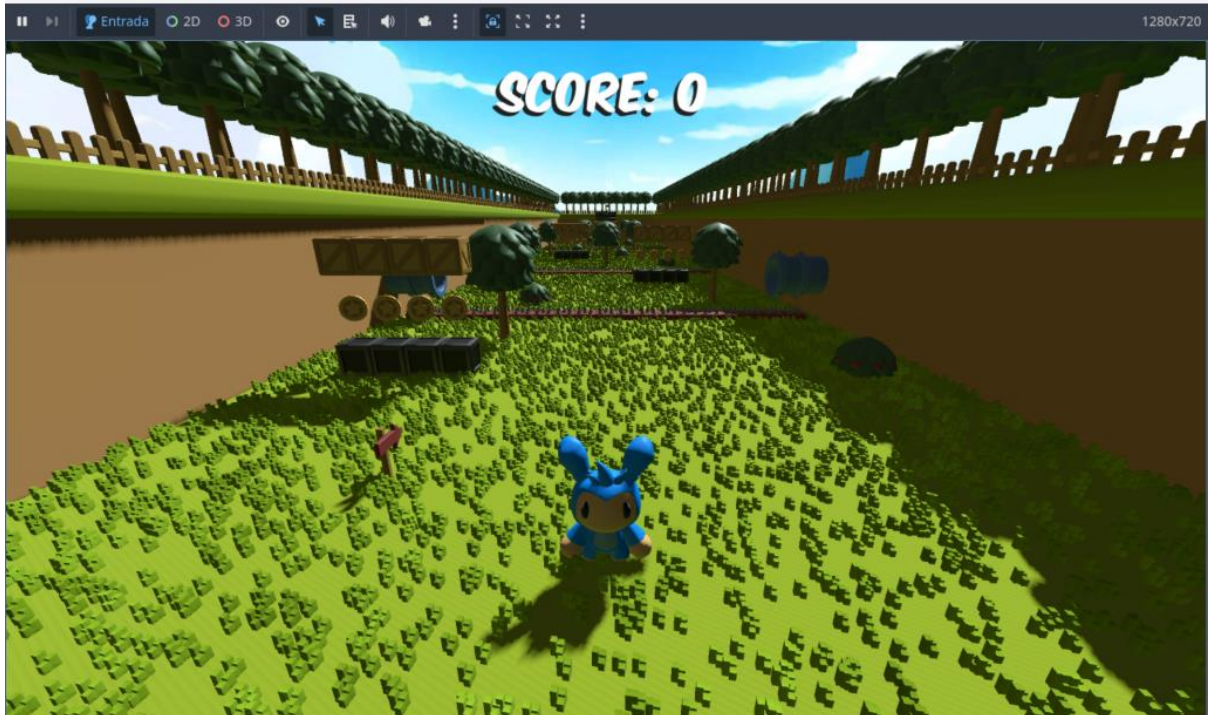


MI PRIMER JUEGO 3D EN GODOT



En esta práctica hemos trabajado con Godot para programar la lógica de un juego de plataformas en 3D. Partimos de un proyecto base ya creado y nos hemos encargado de implementar los distintos scripts que controlan el comportamiento del jugador, las monedas, los obstáculos, la cámara y la interfaz.

Para el desarrollo del juego se han implementado varios scripts, cada uno encargado de controlar una parte específica del funcionamiento. En total, se han programado distintos scripts para gestionar el movimiento del jugador, la cámara, los enemigos, los objetos interactivos como monedas y cajas, la interfaz gráfica y el sistema de puntuación global.

Cada script cumple una función concreta dentro del juego, lo que permite que la lógica esté organizada de forma modular y sea más fácil de entender y mantener. A continuación, se explica el funcionamiento de cada uno de ellos.



Bee.gd:

Este script controla el comportamiento de la abeja como zona de peligro. Su función es detectar cuando el jugador entra en contacto con ella y activar la animación de fin de partida (Game Over).

```
1 extends Area3D
2
3 # Referencia automática a la GUI (CanvasLayer en el grupo "GUI")
4 @onready var gui: CanvasLayer = get_tree().get_first_node_in_group("GUI")
5
6
7 # Se ejecuta cuando un cuerpo entra en el área de la abeja
8 func _on_body_entered(body):
9     # Si el cuerpo es el jugador, lanzamos el Game Over
10     if body is Player:
11         # 3. Lanzamos el FadeOut (Game Over)
12         gui.get_node("FadeScreen/AnimationPlayer").play("FadeOut")
13
14
15
```

Este script hace que la abeja funcione como una zona de peligro. Extiende de Area3D porque solo necesita detectar cuándo el jugador entra en contacto con ella.

Cuando se activa la señal **body_entered**, se comprueba si el objeto que ha entrado es el Player. Si lo es, el script accede a la GUI mediante el sistema de grupos y reproduce la animación **"FadeOut"**, provocando el efecto de Game Over.

Bee_path.gd:

Este script controla el movimiento automático de la abeja siguiendo un recorrido definido en la escena. Extiende de Path3D, ya que utiliza un camino previamente creado para que el objeto se desplace sobre él.

```
1  extends Path3D
2
3  # Velocidad del movimiento (editable desde el Inspector)
4  @export var speed: int = 3
5
6
7  # Se ejecuta en cada frame
8  func _process(delta):
9      # Avanza el objeto por el camino
10     $PathFollow3D.progress += speed * delta
11
```

Se declara una variable speed exportada, lo que permite modificar la velocidad desde el Inspector sin tocar el código. En cada frame, mediante **la función _process(delta), se actualiza el valor de progress del nodo PathFollow3D**, multiplicando la velocidad por delta para que el movimiento sea fluido e independiente de los FPS.

Camera.gd:

Este script hace que la cámara siga al jugador durante la partida. Se le asigna el jugador desde el Inspector y, en cada frame, actualiza su posición en X y Z para moverse junto a él.

Además, fija la altura en el eje Y y añade un pequeño desplazamiento hacia atrás para mantener una vista clara del personaje

```

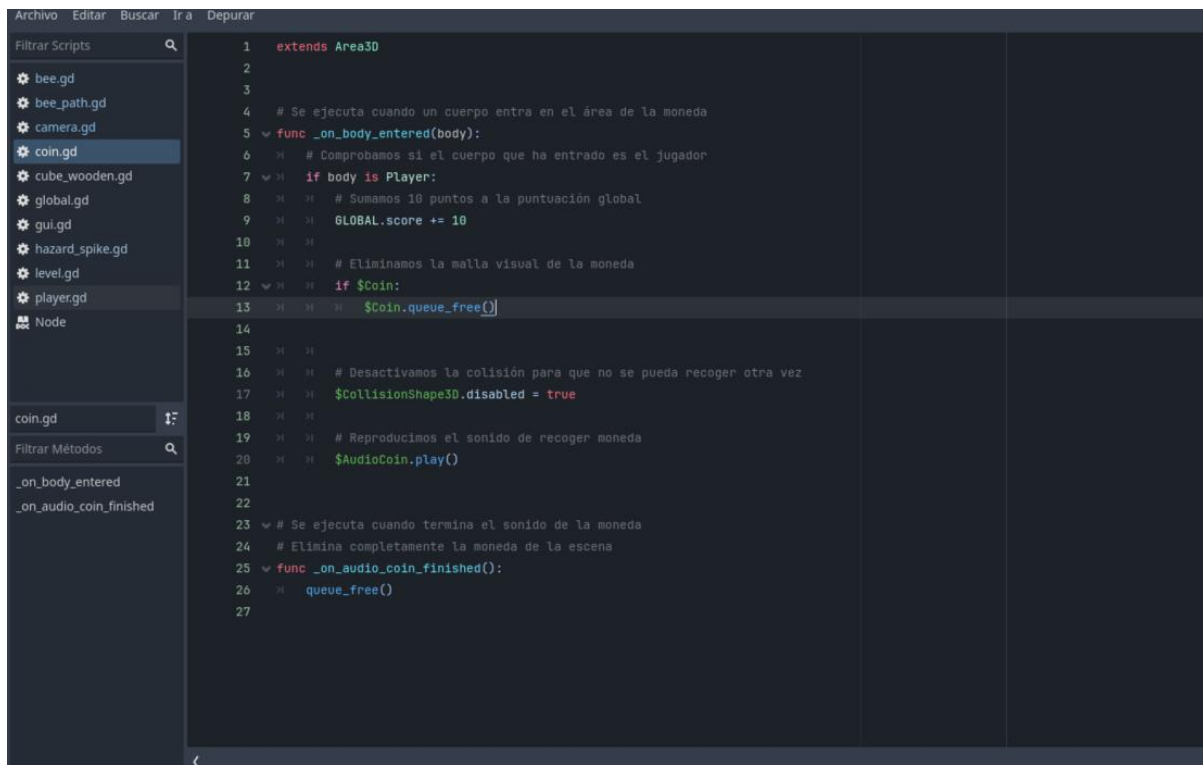
1  extends Camera3D
2
3  # Referencia al jugador (se asigna desde el Inspector)]
4  @export var player: CharacterBody3D
5
6
7  # Se ejecuta al iniciar el nodo
8  func _ready():
9      # Fijamos la altura de la cámara
10     position.y = 9
11
12
13     # Se ejecuta en cada frame
14     func _process(_delta):
15         # Seguimos al jugador en X
16         position.x = player.position.x
17
18         # Seguimos al jugador en Z con un pequeño offset hacia atrás
19         position.z = player.position.z + 10
20

```

Coin.gd:

Este script controla las monedas del juego. Cuando el jugador entra en contacto con una, se suman 10 puntos a la variable global GLOBAL . score, se elimina la parte visual y se desactiva la colisión para que no pueda recogerse otra vez.

Después de reproducir el sonido, la moneda se elimina completamente de la escena.



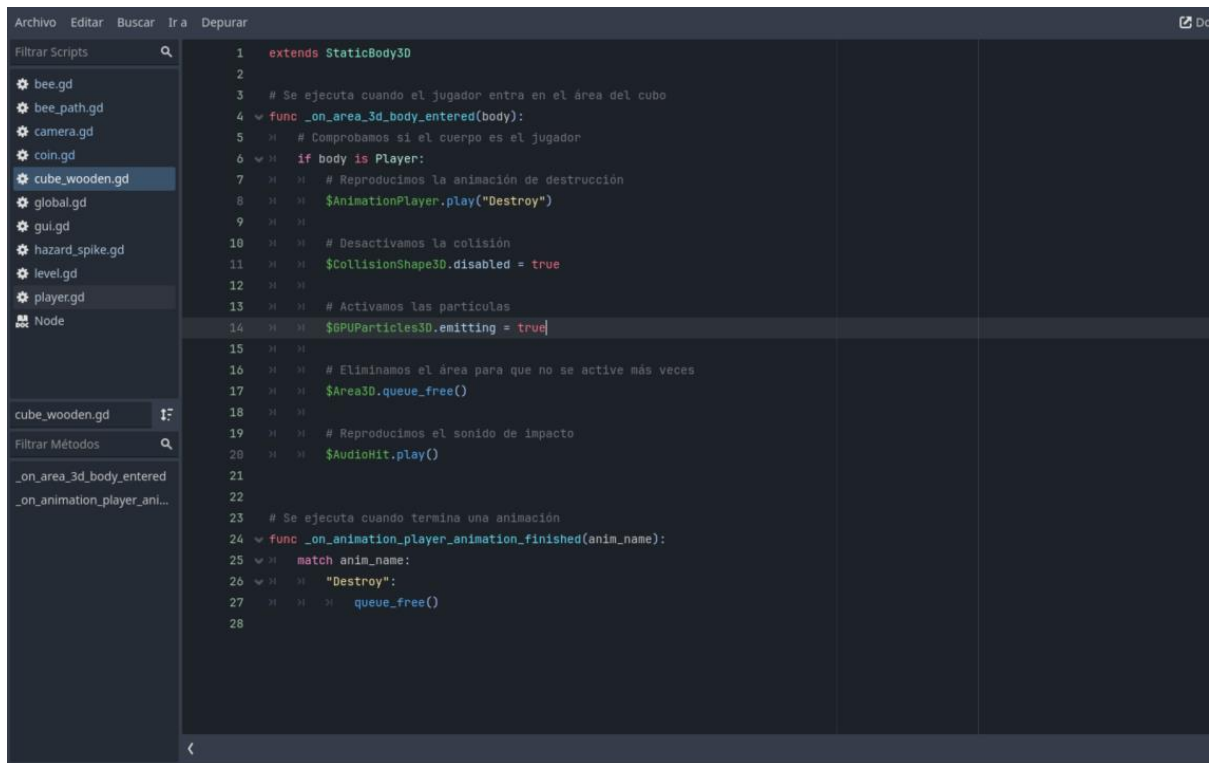
```
1 extends Area3D
2
3
4 # Se ejecuta cuando un cuerpo entra en el área de la moneda
5 func _on_body_entered(body):
6     # Comprobamos si el cuerpo que ha entrado es el jugador
7     if body is Player:
8         # Sumamos 10 puntos a la puntuación global
9         GLOBAL.score += 10
10
11     # Eliminamos la malla visual de la moneda
12     if $Coin:
13         $Coin.queue_free()
14
15
16     # Desactivamos la colisión para que no se pueda recoger otra vez
17     $CollisionShape3D.disabled = true
18
19     # Reproducimos el sonido de recoger moneda
20     $AudioCoin.play()
21
22
23 # Se ejecuta cuando termina el sonido de la moneda
24 # Elimina completamente la moneda de la escena
25 func _on_audio_coin_finished():
26     queue_free()
27
```

Cube_Wooden.gd:

Este script controla el comportamiento de la caja de madera destructible. Extiende de **StaticBody3D** porque es un objeto estático que forma parte del escenario.

Cuando el jugador entra en su área de detección, se reproduce la animación de destrucción, se desactiva la colisión para que el jugador pueda atravesarlo, se activan las partículas y se reproduce el sonido de impacto. Además, se elimina el área para que no pueda activarse más veces.

Cuando termina la animación "**Destroy**", el objeto se elimina completamente de la escena.



Global.gd:

Este script actúa como un **Singleton (Autoload)** y se encarga de almacenar la puntuación global del juego. Extiende de Node porque no necesita representación física en la escena.

Contiene una variable score inicializada en 0, que mantiene la puntuación del jugador aunque se cambie de escena.

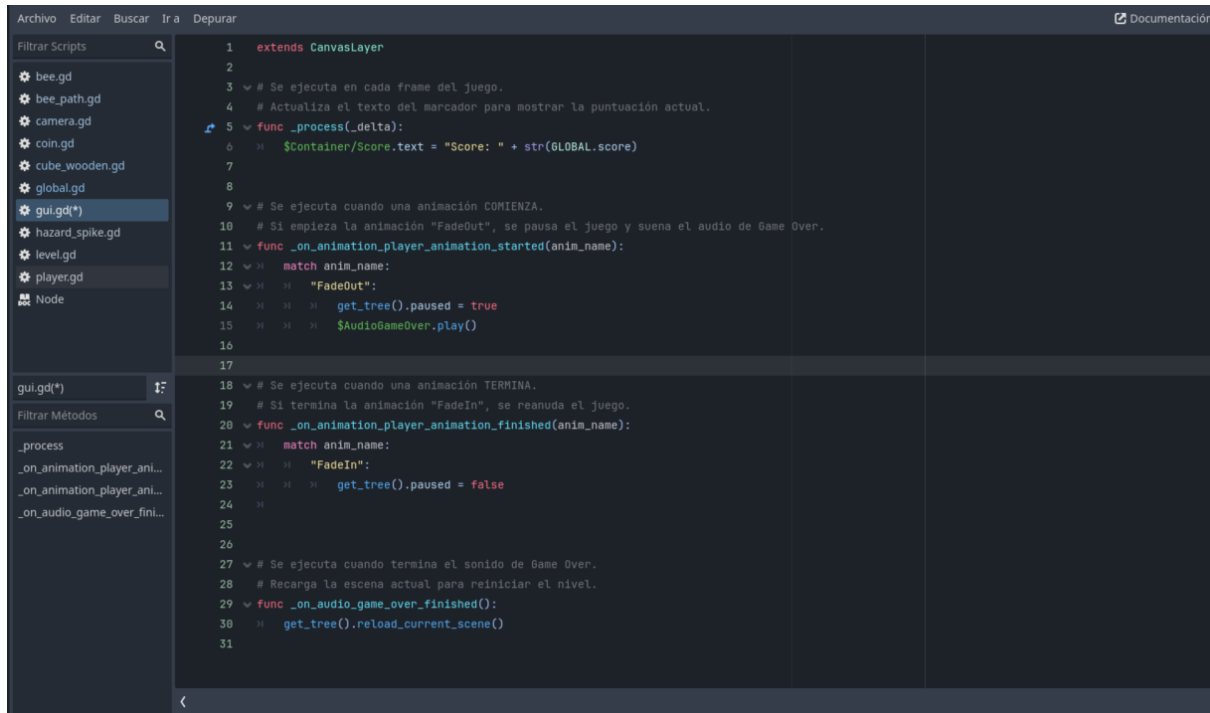
```
1 extends Node
2
3 var score: int = 0
4
```

Guid.gd(*):

Este script controla la interfaz del juego. Extiende de CanvasLayer porque gestiona elementos visuales en 2D como el marcador y las transiciones.

Actualiza en cada frame la puntuación mostrando el valor de GLOBAL.score. Además, controla las animaciones de fundido: cuando comienza "FadeOut" pausa el juego y reproduce el sonido de Game Over, y cuando termina "FadeIn" reanuda la partida.

Por último, cuando finaliza el sonido de Game Over, recarga la escena actual para reiniciar el nivel.

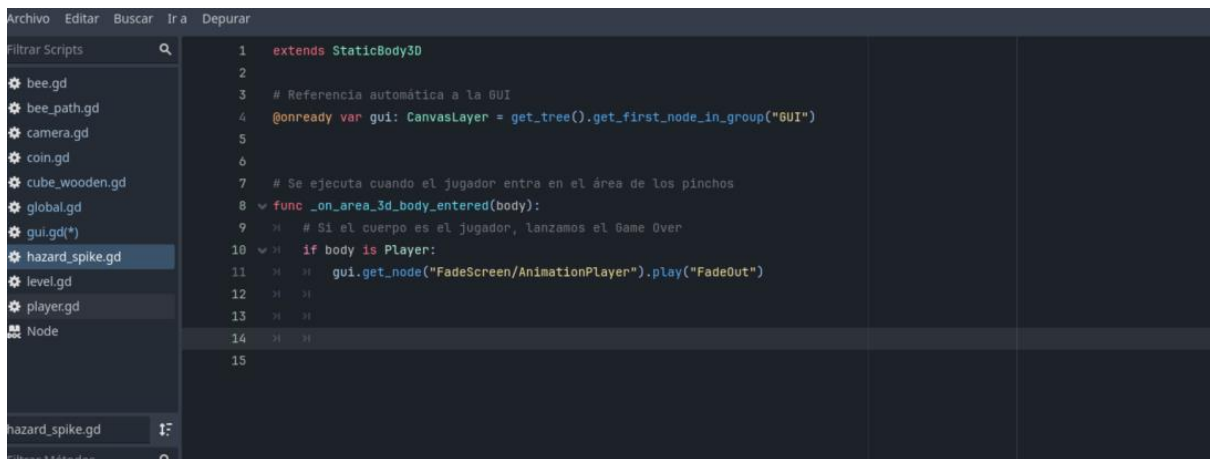


```
1 extends CanvasLayer
2
3 # Se ejecuta en cada frame del juego.
4 # Actualiza el texto del marcador para mostrar la puntuación actual.
5 func _process(delta):
6     $Container/Score.text = "Score: " + str(GLOBAL.score)
7
8
9 # Se ejecuta cuando una animación COMIENZA.
10 # Si empieza la animación "FadeOut", se pausa el juego y suena el audio de Game Over.
11 func _on_animation_player_animation_started(anim_name):
12     match anim_name:
13         "FadeOut":
14             get_tree().paused = true
15             $AudioGameOver.play()
16
17
18 # Se ejecuta cuando una animación TERMINA.
19 # Si termina la animación "FadeIn", se reanuda el juego.
20 func _on_animation_player_animation_finished(anim_name):
21     match anim_name:
22         "FadeIn":
23             get_tree().paused = false
24
25
26
27 # Se ejecuta cuando termina el sonido de Game Over.
28 # Recarga la escena actual para reiniciar el nivel.
29 func _on_audio_game_over_finished():
30     get_tree().reload_current_scene()
31
```

Hazard_Spike.gd:

Este script controla el comportamiento de los pinchos como obstáculo del juego. Extiende de StaticBody3D porque es un objeto fijo del escenario.

Cuando el jugador entra en su área de detección, se comprueba que el cuerpo sea el Player y, si lo es, se accede a la GUI mediante el sistema de grupos para reproducir la animación "FadeOut".



```
1 extends StaticBody3D
2
3 # Referencia automática a la GUI
4 @onready var gui: CanvasLayer = get_tree().get_first_node_in_group("GUI")
5
6
7 # Se ejecuta cuando el jugador entra en el área de los pinchos
8 func _on_area_3d_body_entered(body):
9     # Si el cuerpo es el jugador, lanzamos el Game Over
10     if body is Player:
11         gui.get_node("FadeScreen/AnimationPlayer").play("FadeOut")
12
13
14
15
```

Level.gd:

Este script se encarga de configurar el estado inicial del nivel. Extiende de Node3D porque actúa como nodo principal de la escena.

Cuando se inicia el nivel, reinicia la puntuación estableciendo GLOBAL.score en 0 y pausa el juego hasta que se produzca la animación de inicio.



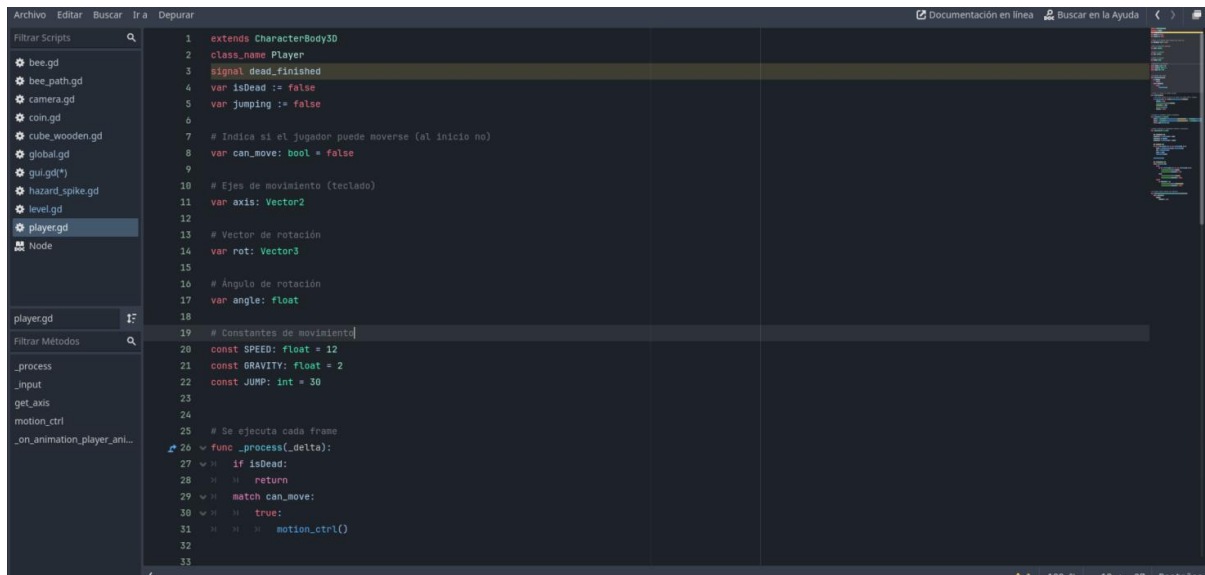
Player.gd:

Este script controla todo el comportamiento del jugador. Extiende de CharacterBody3D porque necesita físicas, gravedad y movimiento mediante move_and_slide().

Gestiona el desplazamiento en los ejes X y Z según las teclas pulsadas, aplicando una velocidad constante (SPEED) y una gravedad progresiva (GRAVITY). Además, permite saltar cuando el jugador está en el suelo, aplicando una fuerza vertical (JUMP) y reproduciendo la animación y el sonido correspondientes.

También calcula automáticamente la rotación del personaje para que mire hacia la dirección en la que se mueve. Dependiendo del estado (quieto, corriendo o saltando), reproduce distintas animaciones y activa o desactiva partículas.

Por último, controla cuándo el jugador puede empezar a moverse (can_move), normalmente después de una animación inicial.



Organización del Proyecto

En este proyecto el juego está organizado en distintas carpetas dentro del directorio res://. Esta estructura permite separar los distintos tipos de archivos (recursos, escenas, scripts, configuración, etc.) para mantener el proyecto limpio y fácil de mantener.

Carpeta Addons:

Esta carpeta se utiliza para incluir complementos o plugins externos que amplían las funcionalidades de Godot.

Normalmente aquí se guardan herramientas adicionales instaladas desde el editor o desarrolladas por terceros. En este proyecto no afecta directamente a la lógica del juego, pero forma parte de la estructura estándar de Godot.

Carpeta Assets:

La carpeta assets contiene todos los recursos gráficos y modelos 3D utilizados en el juego. Aquí se guardan los elementos visuales que forman parte del escenario y de los objetos interactivos.

Dentro de esta carpeta encontramos varias subcarpetas organizadas por tipo de elemento:

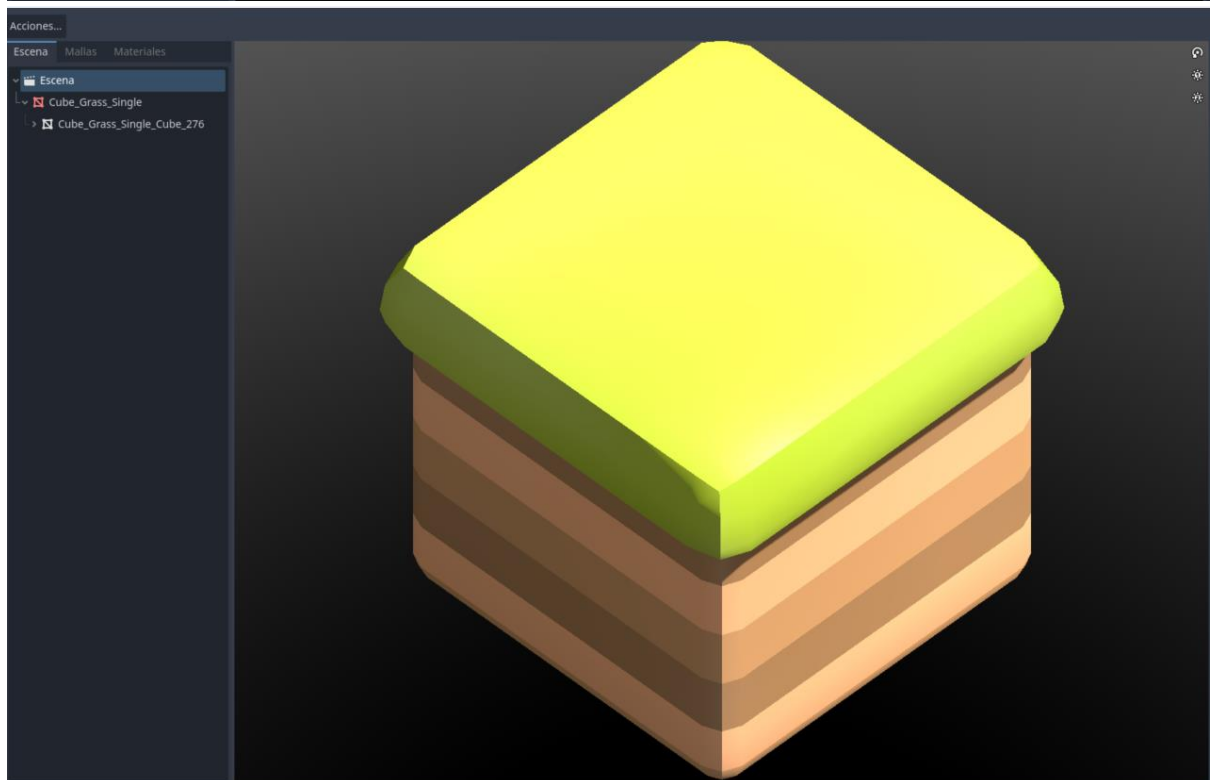
Character

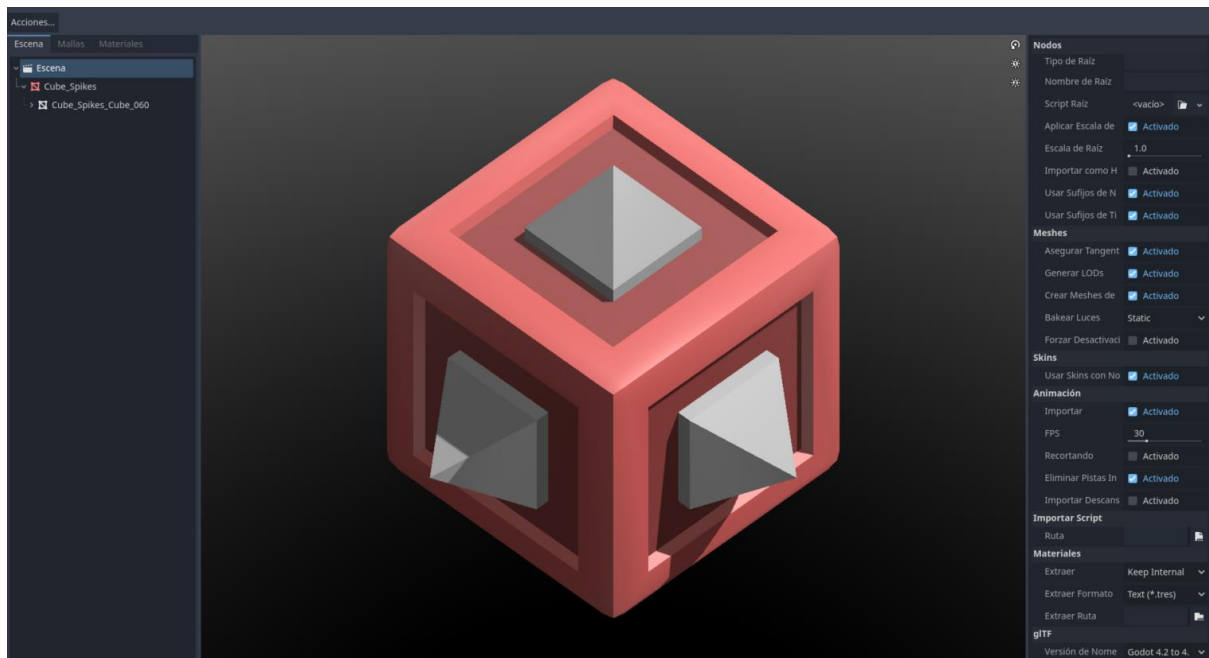
Contiene los modelos y recursos relacionados con el personaje principal (mallas, animaciones, materiales, etc.).



Cubes

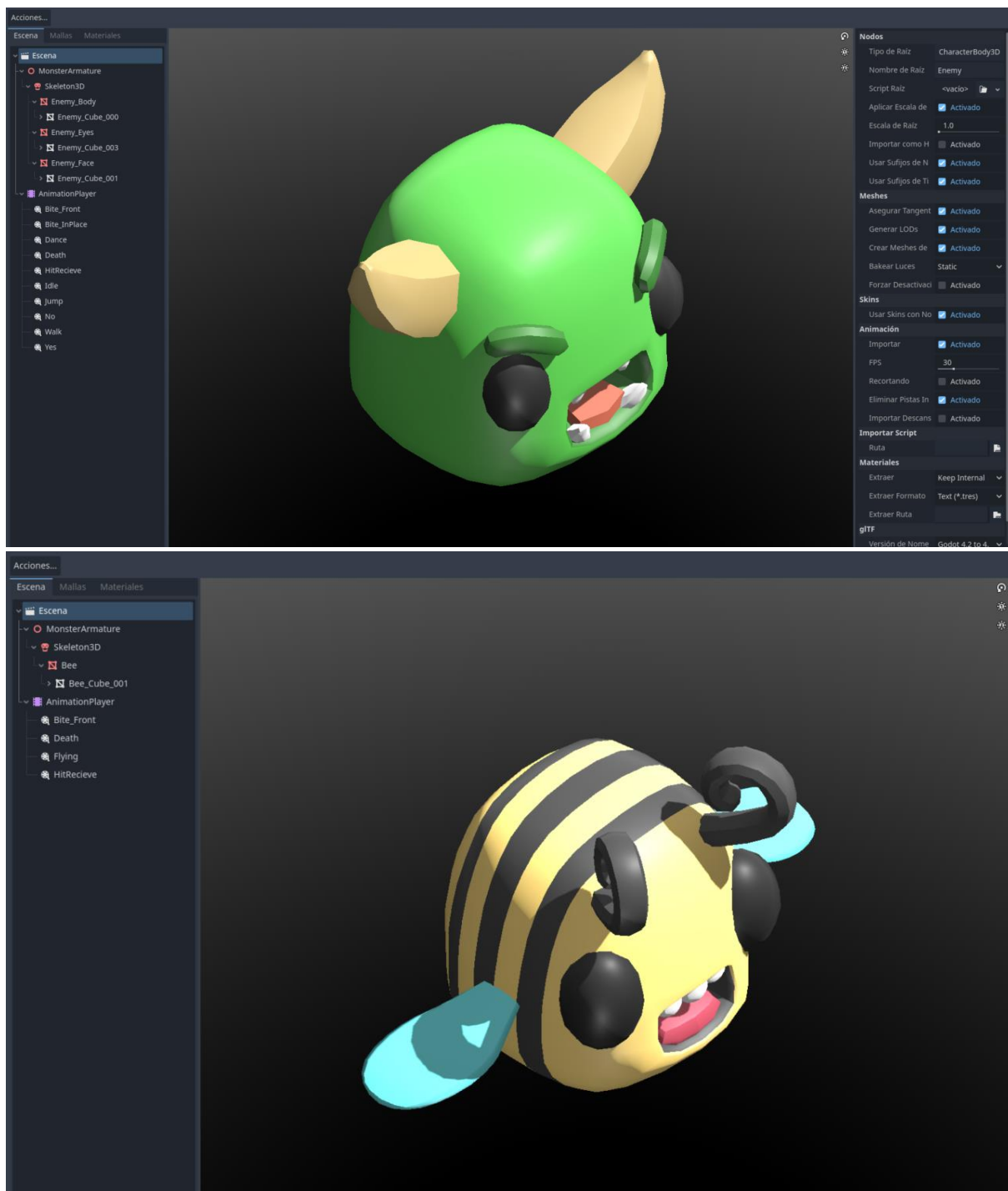
Incluye los modelos de cubos u objetos similares que forman parte del entorno, como las cajas destructibles.





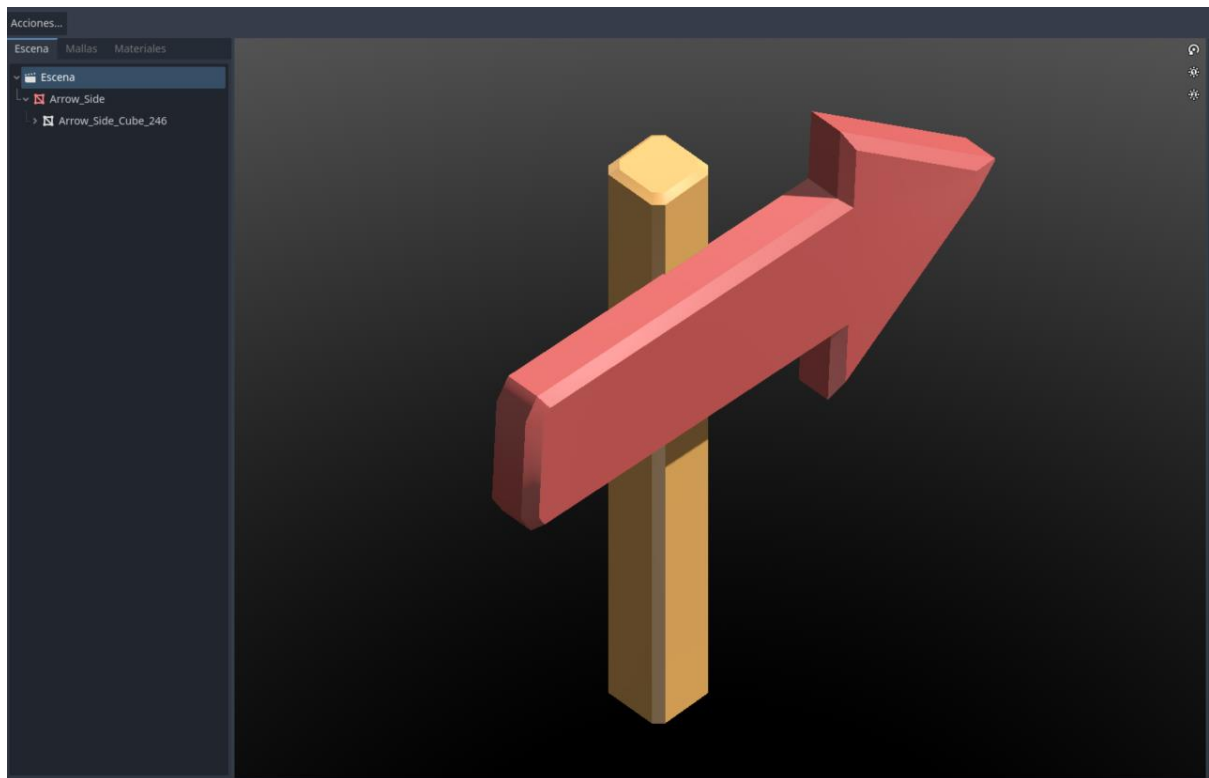
Enemies

Guarda los modelos y recursos visuales de los enemigos del juego, como la abeja.



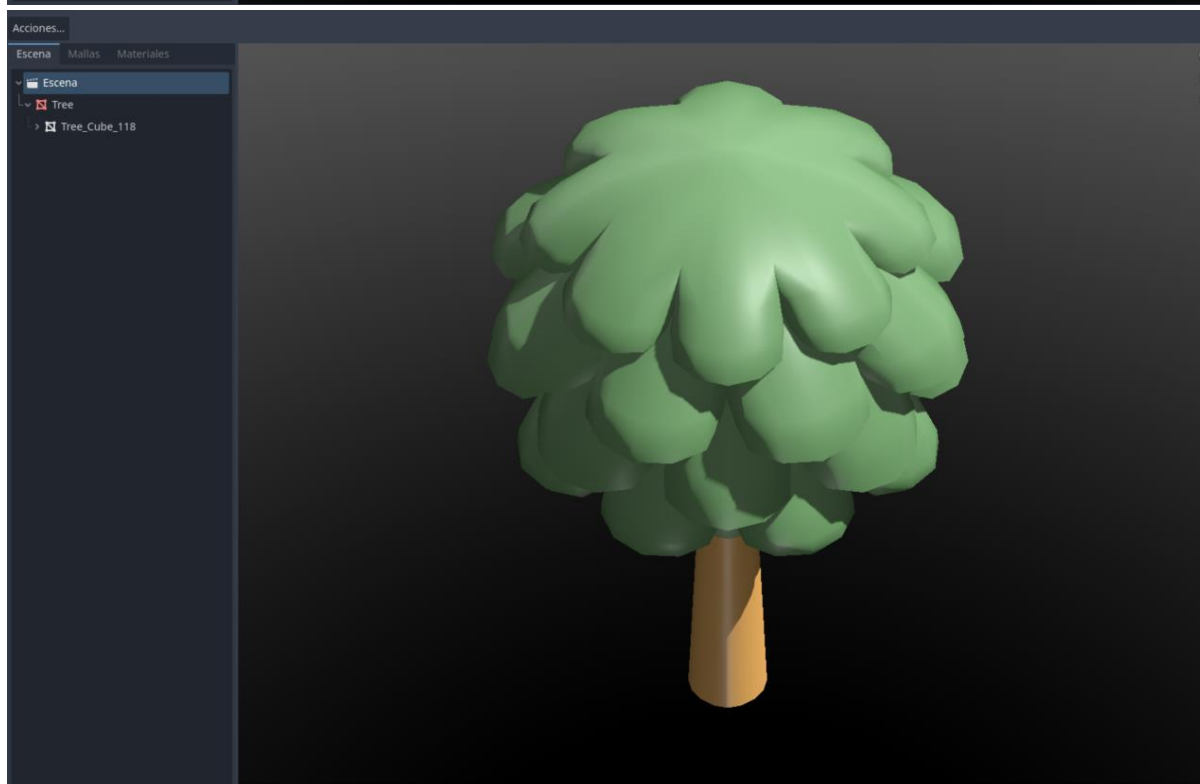
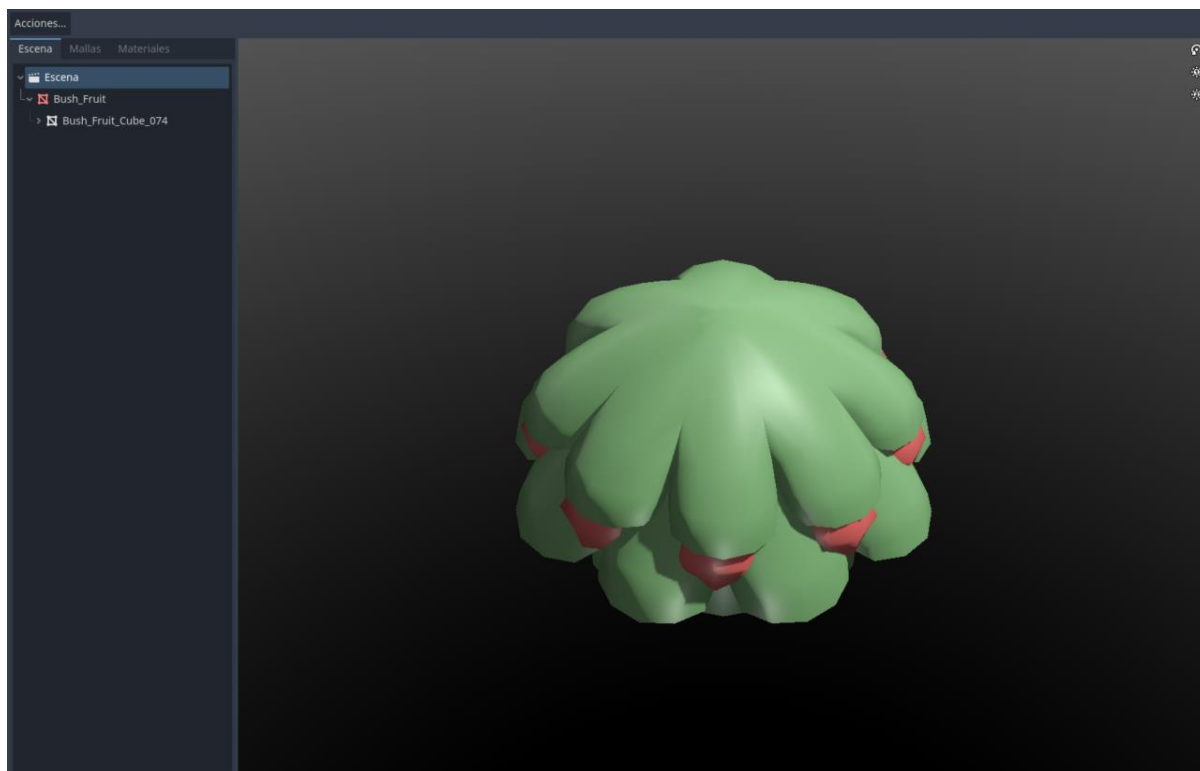
LevelAndMechanics

Contiene elementos relacionados con la estructura del nivel y mecánicas específicas del juego.



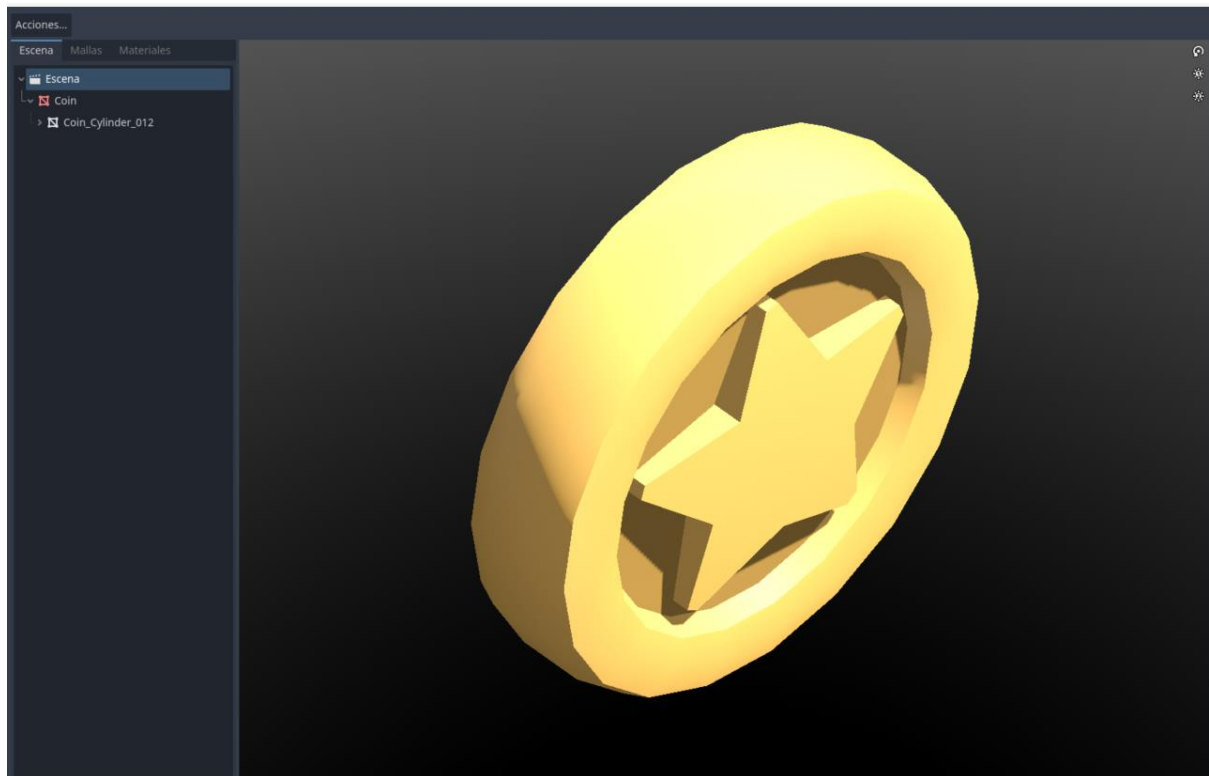
Nature

Incluye objetos decorativos del entorno natural, como árboles, plantas o elementos del paisaje.



PowerUpsAndPickUps

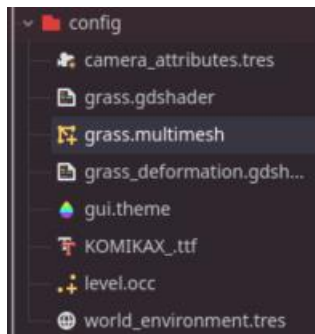
Guarda los objetos que el jugador puede recoger, como monedas o potenciadores.



Carpeta Config:

La carpeta config contiene archivos de configuración del proyecto. Aquí se guardan datos relacionados con ajustes internos del juego, como parámetros técnicos o configuraciones que no forman parte directamente de la lógica ni de los elementos visuales.

Su función principal es separar la configuración del resto del contenido (scripts, escenas y recursos), lo que ayuda a mantener el proyecto más organizado.

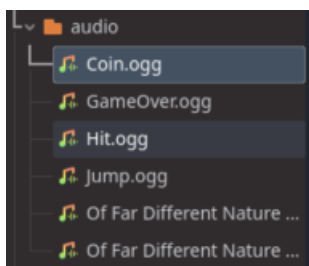


Carpeta Resources:

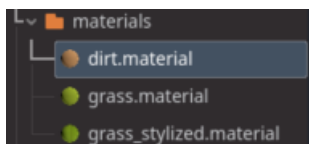
La carpeta resources guarda archivos necesarios para el funcionamiento del juego, como sonidos, materiales y scripts.

Dentro encontramos:

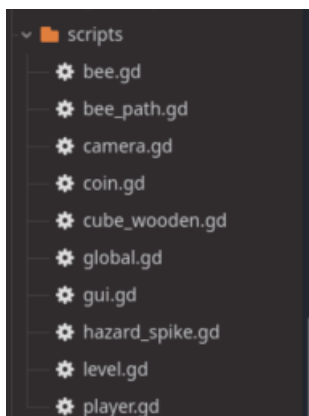
- audio: donde están los sonidos del juego.



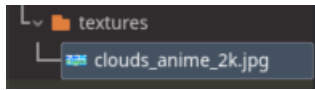
- materials: donde se guardan los materiales visuales.



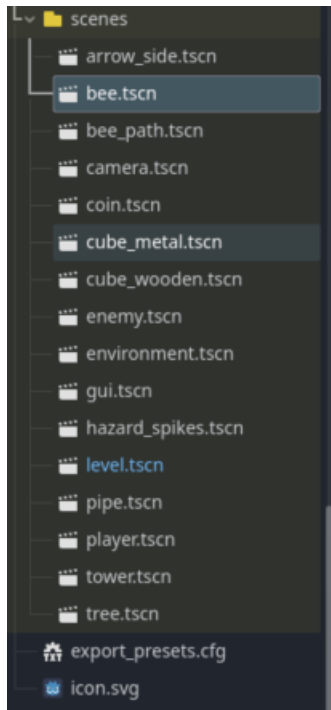
- scripts: donde están todos los archivos .gd que controlan la lógica del juego (player, coin, gui, etc.).



- textures: imágenes utilizadas como texturas para dar apariencia a los objetos



Carpeta Scenes:



La carpeta scenes contiene las distintas escenas del juego. En Godot, una escena es una estructura que agrupa nodos organizados para formar una parte del juego, como el nivel principal, el jugador o la interfaz.

Aquí se guardan los archivos .tscn, que representan los niveles y elementos principales del proyecto ya montados visualmente.

En resumen, esta carpeta contiene la estructura del juego ya construida, mientras que los scripts dentro de resources/scripts se encargan de darle funcionamiento.

CONCLUSION:

En esta práctica he aprendido cómo se organiza un proyecto en Godot y cómo se programa la lógica de un juego en 3D mediante distintos scripts. He trabajado con movimiento, colisiones, animaciones y variables globales.

En general, me ha ayudado a entender mejor cómo funciona un videojuego desde el punto de vista de la programación.

